

Model Report: Bi-LSTM Siamese Encoder (Built From Scratch)

Milestone 4 requirement — "one model built from scratch" Project: Smart MCQ Solver Challenge

1. Motivation

The TF-IDF cosine-similarity baseline (see `TFIDF_MODEL_SUMMARY.md`) scored **below random guessing** ($\text{MAP@3} = 0.280$ vs. ≈ 0.367 random baseline). Root-cause analysis showed the 5 answer options per question are near-paraphrases of each other, sharing 70–90% of their vocabulary — so raw lexical overlap between prompt and option carries almost no signal.

This motivates a model that can **learn** semantic patterns from the training data rather than relying on hand-crafted word-overlap statistics. A Bi-LSTM Siamese Encoder is the natural next step: it processes word order and sequence structure, and its parameters are optimized directly against the task instead of being fixed a priori.

2. Architecture

Fully from scratch — no pretrained weights, no pretrained embeddings (e.g. GloVe/Word2Vec) anywhere. `nn.Embedding` is randomly initialized and trained end-to-end.

```
prompt text → tokenize → [word ids] ─┐
                                     │→ shared Bi-LSTM encoder → pooled vector
option A/B/C/D/E text → tokenize → [word ids] ─┘ (weights shared across all 6 inputs)

match_features = concat[prompt_vec, option_vec, prompt_vec ⊙ option_vec, |prompt_vec - option_vec|]
score = MLP(match_features) # one scalar per option

logits = [score_A, score_B, score_C, score_D, score_E]
prediction = softmax(logits), top-3 taken for MAP@3
```

Components

Component	Detail
Vocabulary	Built from training text only; word-frequency based, <code><pad></code> / <code><unk></code> tokens, capped at 20,000 words
Embedding	<code>nn.Embedding(vocab_size, 128)</code> , randomly initialized, trainable
Encoder	Single-layer Bi-LSTM, hidden size 128 (→ 256-dim pooled output after concatenating both directions' final hidden states)

Component	Detail
Weight sharing	Siamese — the same encoder processes the prompt and all 5 options, no separate parameters per option slot
Matching head	4-way feature concatenation (<code>prompt</code> , <code>option</code> , elementwise product, absolute difference) → Linear(1024→128) → ReLU → Dropout → Linear(128→1)
Output	5-way softmax per question, trained with cross-entropy against the correct label index
Regularization	Dropout 0.3 (embedding + LSTM output + matching head), weight decay 1e-5, gradient clipping (max norm 5.0)
Parameters	~776,000 trainable

3. Training Configuration

Hyperparameter	Value
Embedding dim	128
Hidden dim	128 (Bi-LSTM → 256 effective)
Batch size	32
Optimizer	Adam, lr = 1e-3
Epochs	15
Loss	Cross-entropy (5-way)
Max prompt length	40 tokens
Max option length	60 tokens
Random seed	42 (fixed for reproducibility)

4. A Data Leakage Bug Found and Fixed Before Trusting Results

Initial run (naive random 80/20 stratified split) produced a suspiciously high **validation MAP@3 = 0.996**.

Investigation: cross-checked train/validation splits after stripping the instruction-wrapper phrasing from each prompt (e.g. *"Pick the best possible answer: ... among the listed options."* → core question only). Result: **201 / 400 validation rows (50.2%) had a near-duplicate "twin" question sitting in the training split**. The model wasn't generalizing — it was pattern-matching validation questions to memorized training duplicates.

This directly follows from the EDA finding (see `EDA_SUMMARY.md`) that 1,227 / 2,000 training rows share a core question with at least one other row after wrapper-stripping.

Fix: replaced the naive split with `StratifiedGroupKFold` , grouping by core question so no near-duplicate can appear in both train and validation. Verified programmatically that core-prompt overlap between the two splits is exactly zero before trusting any subsequent metric.

5. Results (Leakage-Free Validation Split)

Metric	Score
Best Validation Accuracy (Top-1)	0.713

Metric	Score
Best Validation MAP@3	0.773

Comparison across models so far

Model	Validation MAP@3
Random baseline	≈0.367
TF-IDF + Cosine Similarity	0.280
Bi-LSTM Siamese Encoder (this model)	0.773

The Bi-LSTM more than doubles the TF-IDF baseline's score and clears the random baseline by a wide margin — evidence that a learned, order-aware representation captures signal that pure lexical overlap cannot.

6. Training Behavior & Known Limitation

- Training loss decreases smoothly and monotonically toward ~0.04 by epoch 15.
- Validation loss bottoms out around **epoch 8–10**, then fluctuates and trends upward — a clear **overfitting** signature on a small (1,600-row) training set.
- Validation MAP@3 continues to inch upward through epoch 14 despite rising validation loss, showing the ranking quality is more stable than the raw loss value — but this divergence is still a sign the model is starting to memorize training-specific patterns rather than improving generalizable understanding.

Mitigation already in place: the checkpoint saved is the one with the **best validation MAP@3 across all epochs**, not the final epoch — so the overfit late-stage weights are not what gets deployed.

Further mitigations for future iterations:

- Early stopping (halt once val MAP@3 plateaus for N epochs)
 - Increase dropout or weight decay
 - Reduce hidden/embedding dimensionality given the small dataset size
 - Data augmentation (e.g. paraphrasing, option shuffling) to increase effective training set size
-

7. Why This Model Satisfies the "Built From Scratch" Requirement

- No pretrained checkpoints, embeddings, or tokenizers are used anywhere in the pipeline.
- The vocabulary, embedding table, and all LSTM/MLP weights are initialized randomly and learned purely from this competition's training data.
- Architecture, training loop, evaluation, and leakage-safe splitting were all implemented directly (PyTorch primitives: `nn.Embedding`, `nn.LSTM`, `nn.Linear`), not imported from a library-level pre-built model class.

8. Artifacts

File	Description
04_bilstm_scratch_model.py	Full training + inference script
bilstm_scratch_model.pt	Saved weights, vocabulary, and config (best checkpoint by val MAP@3)
submission_bilstm.csv	Kaggle-ready submission, format-validated
eda_outputs/bilstm_training_curves.png	Loss and metric curves across training
W&B run	Project 23f3003691-t22026 , run BiLSTM_Scratch_Baseline

9. Next Steps

1. Train the **pretrained model** (Milestone 2: BERT/RoBERTa via Hugging Face `transformers`) as the second required model, and compare against this from-scratch baseline on the same leakage-free validation split.
2. Consider ensembling the Bi-LSTM and TF-IDF signals as an additional model-of-choice, since they may make different kinds of errors.
3. Apply the same `StratifiedGroupKFold` leakage-safe splitting strategy to all future models in this project — this is now a standing methodological requirement, not optional.